Lovely Professional University

CSM422 · DEEP LEARNING

Continuous Assessment 1 · Project Report

EnergySense

Household Electricity Consumption Forecasting Using an Artificial Neural Network

One-minute-ahead prediction of global active power with a feedforward TensorFlow/Keras network, evaluated against a persistence baseline

Submitted by	Kartik
Registration number	12319010
Section	K23UP
Course instructor	Karan Kumar Das
Date of submission	October 2026

Code: https://github.com/Kartik070704/Deep-Learning

Dashboard: https://deep-learning-2q62ytadkeqmj3cjkotrun.streamlit.app/

Abstract

This report presents EnergySense, a deep-learning project that forecasts the global active power of a household one minute ahead. The work uses the real Individual Household Electric Power Consumption dataset, which contains 2,075,259 one-minute readings of which 2,049,280 are complete. A chronological sample of 50,000 readings is used, and no value is imputed or synthesised.

Twelve leakage-free features are engineered from strictly earlier readings: six electrical measurements one minute earlier, active power at one and thirty minutes earlier, and cyclic hour and weekday encodings. A feedforward network with layers of 64 and 32 ReLU neurons (2,945 parameters) is trained with Adam and early stopping on the first 70% of the data. It predicts a residual correction to the previous reading, scaled by a factor calibrated on a 15% validation block. The final 15% (7,495 minutes) is held out for testing.

On the test block the ANN achieves an MAE of 0.1596 kW, an RMSE of 0.3717 kW and an R^2 of 0.8984. The persistence baseline, which repeats the previous minute, achieves 0.1408 kW, 0.3850 kW and 0.8910. The ANN is therefore better on RMSE and R^2 but worse on MAE. Quantile, residual and hourly analyses show that it reduces the largest errors while being slightly less accurate than persistence on typical minutes. The system is delivered as a reproducible code base and a deployed Streamlit dashboard for data exploration, results review and live prediction.

Keywords: *household electricity, load forecasting, artificial neural network, multilayer perceptron, persistence baseline, time series, Streamlit.*

Contents

1. Introduction **5**
1.1 Background 5
1.2 Problem statement 5
1.3 Objectives 5
1.4 Scope and contributions 5
1.5 Organisation of the report 6
2. Theoretical Background **6**
2.1 The artificial neuron and activation functions 6
2.2 Feedforward propagation 6
2.3 Loss function and evaluation metrics 6
2.4 Backpropagation and the Adam optimiser 6
2.5 Early stopping 7
2.6 Forecasting principles: persistence, leakage and chronological splits 7
3. Dataset **7**
3.1 Source and description 7
3.2 Data quality and cleaning 8
3.3 The selected experimental sample 8
3.4 Exploratory analysis 9
4. Methodology **10**
4.1 Supervised formulation 10
4.2 Feature engineering 10
4.3 Leakage prevention and exact lag matching 11
4.4 Chronological train, validation and test split 11
4.5 Feature scaling 11
4.6 Network architecture 12
4.7 Residual formulation and validation calibration 12
4.8 Training configuration 13
4.9 Evaluation protocol 13
5. Results and Analysis **13**
5.1 Training behaviour 13
5.2 Headline test results 14
5.3 Actual versus predicted values 15
5.4 Residual analysis 15
5.5 Forecasts over time 16
5.6 Distribution of absolute error 16
5.7 Error by hour of day 17
5.8 Summary of findings 17
6. System Implementation **18**
6.1 Software stack and project structure 18
6.2 The shared feature module 18
6.3 The Streamlit dashboard 18
6.4 Prediction pipeline 19
6.5 Testing and reproducibility 19
6.6 Dashboard screenshots 19
7. Discussion **21**
7.1 Interpreting the mixed result 21
7.2 Limitations 22
7.3 Threats to validity 22
7.4 Ethical and privacy considerations 22
8. Conclusion and Future Work **22**
8.1 Conclusion 22
8.2 Future work 23
References **23**
Appendix A: Reproduction Commands **23**
Appendix B: Key Code Excerpts **24**
Appendix C: Raw Evaluation Output **25**

List of Figures

Figure 1: Global active power over the selected five-week sample 9
Figure 2: Distribution of global active power in the sample 9
Figure 3: Average load by hour of day 9
Figure 4: Correlation matrix of the seven measurements 10
Figure 5: Chronological partitioning of the supervised samples 11
Figure 6: Architecture of the EnergySense multilayer perceptron 12
Figure 7: Training and validation loss (MSE) per epoch 13
Figure 8: MAE, RMSE and R^2 for the ANN and the persistence baseline 14
Figure 9: Actual versus ANN-predicted active power on the test set 15
Figure 10: Distribution of test residuals (actual − prediction) 15
Figure 11: Actual and ANN forecast over a chronological sample of the test period 16
Figure 12: Detail of the first 180 test minutes with the persistence forecast added 16
Figure 13: Cumulative distribution of absolute error for the two forecasters 17
Figure 14: Mean absolute error of both forecasters by hour of day 17
Figure 15: Dashboard page: dataset counts, forecast-performance chart and the four evaluation plots 20
Figure 16: Prediction page: 30-row input table, CSV upload and the estimate button 20
Figure 17: Data Explorer page: date range, measurement chart, statistics and table 21
Figure 18: About page: model workflow and responsible-interpretation notes 21

List of Tables

Table 1: Fields of the dataset 7
Table 2: Row counts through the pipeline 8
Table 3: Descriptive statistics of the 50,000-row sample (last column: all valid rows) 8
Table 4: The twelve input features 10
Table 5: Partition sizes 11
Table 6: Layer structure and parameter count 12
Table 7: Training hyperparameters 13
Table 8: Selected points of the training history 13
Table 9: Test-set performance on 7,495 held-out minutes 14
Table 10: Residual summary (actual − forecast, kW) 15
Table 11: Quantiles of the absolute error on the test set (kW) 16
Table 12: Repository contents 18

1. Introduction

1.1 Background

Electricity is one of the largest and most variable running costs of a household, and the way a home draws power changes from minute to minute as appliances switch on and off. Smart meters now record this behaviour at high resolution, which makes it possible to study consumption as a time series and to build models that anticipate it. Short-horizon forecasts of household load support several practical goals: giving residents feedback about their usage, scheduling flexible appliances, detecting unusual consumption, and helping utilities understand demand at the edge of the network.

Classical statistical methods such as persistence, moving averages and ARIMA remain strong baselines for short-term forecasting. Artificial neural networks (ANNs) offer a complementary approach: given informative inputs, a network can learn non-linear relationships between recent measurements, the time of day and the next value, without these relationships being specified by hand. This project, **EnergySense**, applies a small feedforward neural network, also called a multilayer perceptron (MLP), to one-minute-ahead forecasting of household active power, and evaluates it honestly against the simplest possible forecaster.

1.2 Problem statement

Given the electrical measurements recorded in the minutes before time t, together with the time of day and day of the week, estimate the household global active power at time t (in kilowatts). The forecast horizon is exactly one minute. The task is a supervised regression problem on a real, irregularly behaving signal, and the central question of the report is whether a trained ANN can improve on the persistence forecast, which simply repeats the previous minute's reading.

1.3 Objectives

1. Inspect and clean a real household electricity dataset without inventing or imputing any observations.
2. Formulate a leakage-free, one-minute-ahead supervised learning problem with exact timestamp-based lag features.
3. Design, train and regularise a feedforward ANN in TensorFlow/Keras using a strictly chronological train/validation/test split.
4. Evaluate the model on held-out data using MAE, RMSE and R^2, and compare it with a persistence baseline.
5. Analyse where the model helps and where it does not, using residual, quantile and time-of-day diagnostics.
6. Deliver the work as a reproducible code base and an interactive Streamlit dashboard for exploration and prediction.

1.4 Scope and contributions

The work uses one publicly available household dataset and a deliberately compact network (12 inputs, hidden layers of 64 and 32 neurons, one output). The contributions are: (i) a complete, reproducible pipeline from raw text file to trained model; (ii) a residual formulation in which the network learns a correction to the persistence forecast, which makes the comparison with the baseline direct; (iii) a set of diagnostics that go beyond a single headline number; and (iv) a deployed dashboard that exposes the

data, results and a live prediction form. All numbers reported here are measured by the project scripts on the held-out test partition; none are estimated or assumed.

1.5 Organisation of the report

Section 2 reviews the neural-network and forecasting concepts used. Section 3 describes and explores the dataset. Section 4 presents the methodology: features, split, architecture and training. Section 5 reports the results with all evaluation figures. Section 6 describes the software and dashboard. Section 7 discusses the findings, limitations and ethical aspects, and Section 8 concludes with future work. The appendices list the repository contents, key code excerpts and the raw metric file.

2. Theoretical Background

2.1 The artificial neuron and activation functions

An artificial neuron computes a weighted sum of its inputs plus a bias and passes the result through a non-linear activation function. For an input vector x, weight vector w and bias b:

$$z = w^{T}x + b, \qquad a = f(z)$$

The activation used in both hidden layers of EnergySense is the rectified linear unit (ReLU), f(z) = max(0, z). ReLU is cheap to compute, does not saturate for positive inputs and typically trains faster than sigmoid or tanh units in feedforward regression networks (Nair and Hinton, 2010). The output neuron uses a linear (identity) activation so that the network can produce any real-valued number, which is appropriate for regression.

2.2 Feedforward propagation

In a feedforward network the activations of one layer become the inputs of the next. For layer l with weight matrix $W^{(l)}$ and bias vector $b^{(l)}$, the forward pass is:

$$a^{(l)} = f(\, W^{(l)}\, a^{(l-1)} + b^{(l)}\,), \qquad a^{(0)} = x$$

There are no cycles or recurrent connections: information flows from the 12 input features, through the two hidden layers, to a single output. Because the network sees only a fixed set of engineered lag features, any temporal memory must be supplied through those inputs rather than through the architecture itself.

2.3 Loss function and evaluation metrics

Training minimises the mean squared error (MSE) between the target and the network output. MSE penalises large errors quadratically, which suits a signal with occasional sharp peaks. Model quality on unseen data is reported with three complementary metrics, where y_i is the observed value, $\hat{y}_i$ the forecast, $\bar{y}$ the mean of the observations and n the number of test minutes:

$$MSE = (1/n)\, \Sigma\, (y_i - \hat{y}_i)^2, \qquad RMSE = \sqrt{MSE}$$

$$MAE = (1/n)\, \Sigma\, |y_i - \hat{y}_i|, \qquad R^2 = 1 - \Sigma (y_i - \hat{y}_i)^2 \,/\, \Sigma (y_i - \bar{y})^2$$

MAE is the average size of the error in kilowatts and is easy to interpret. RMSE weights large errors more heavily and therefore reflects how well a model handles sudden jumps in load. R^2 expresses the share of variance explained. Because MAE and RMSE respond differently to the shape of the error distribution, a model can win on one and lose on the other; this report presents both so that neither is hidden.

2.4 Backpropagation and the Adam optimiser

Learning adjusts the weights and biases to reduce the loss. Backpropagation applies the chain rule layer by layer, from the output back to the input, to compute the gradient of the loss with respect to every parameter (Rumelhart, Hinton and Williams, 1986). A gradient-based optimiser then updates the parameters. EnergySense uses Adam (Kingma and Ba, 2015), which keeps running estimates of the first and second moments of the gradients and gives each parameter its own adaptive step size. Adam is a robust default for small dense networks and needs little manual tuning.

2.5 Early stopping

A network can keep lowering its training loss while its performance on new data stops improving or worsens. Early stopping monitors the loss on a validation set and halts training once it has failed to improve for a set number of epochs (the patience), then restores the weights from the best epoch (Prechelt, 1998). It is a simple and effective regulariser and removes the need to choose the number of epochs in advance.

2.6 Forecasting principles: persistence, leakage and chronological splits

The **persistence** (or naive) forecaster predicts that the next value equals the most recent one. For a signal that changes slowly relative to its sampling interval it is surprisingly hard to beat, and it is the standard first benchmark in forecasting (Hyndman and Athanasopoulos, 2021). In this dataset the lag-1 autocorrelation of active power is about 0.95, so persistence starts from a very strong position.

Two methodological hazards are specific to time series. The first is **leakage**: any feature that contains information from the target time or later makes results look better than they would be in real use. The second is **random shuffling**: splitting rows at random lets the model train on minutes surrounding the test minutes. EnergySense avoids both by building features only from strictly earlier timestamps and by splitting the rows into consecutive training, validation and test blocks in time order.

3. Dataset

3.1 Source and description

The data are the Individual Household Electric Power Consumption dataset (Hebrail and Berard, 2006) from the UCI Machine Learning Repository. It contains measurements from a single household near Paris, recorded once per minute between December 2006 and November 2010. The file supplied for this project has 2,075,259 rows and nine columns, stored as semicolon-separated text with the character "?" marking missing readings. The original file is read directly by pandas and is never modified.

Table 1: Fields of the dataset

Field	Unit	Description
Date, Time	—	Day-first date (dd/mm/yyyy) and clock time (hh:mm:ss) of the reading
Global_active_power	kW	Household minute-averaged active power (the forecast target)
Global_reactive_power	kW	Household minute-averaged reactive power
Voltage	V	Minute-averaged supply voltage
Global_intensity	A	Minute-averaged current intensity
Sub_metering_1	Wh	Energy in the kitchen (dishwasher, oven, microwave)

Field	Unit	Description
Sub_metering_2	Wh	Energy in the laundry room (washing machine, dryer, refrigerator, light)
Sub_metering_3	Wh	Energy of the electric water heater and air conditioner

3.2 Data quality and cleaning

Inspection found 25,979 rows (1.25% of the file) in which every measurement is missing, leaving 2,049,280 complete readings. The cleaning step parses the timestamps, converts the measurement columns to numbers, drops rows with an invalid timestamp or any missing measurement, sorts the rows by time and keeps the first record when a timestamp is duplicated. No value is imputed, interpolated or synthesised; a missing minute simply stays missing, and any sample that would need it as an input is excluded later.

Table 2: Row counts through the pipeline

Stage	Rows	Share of raw
Raw rows in source file	2,075,259	100.00%
Rows with missing measurements removed	25,979	1.25%
Complete valid readings	2,049,280	98.75%
Selected chronological sample	50,000	2.41%
Supervised samples after lag matching	49,962	2.41%

3.3 The selected experimental sample

Training on all two million rows is not required for a compact network and would make iteration slow, so the experiment uses the first 50,000 complete observations in chronological order. The sample runs from 2006-12-16 17:24:00 to 2007-01-20 10:48:00, a window of about five weeks that spans the Christmas and New Year period. It preserves exactly the nine original columns. Because it is a contiguous block rather than a random subset, the temporal structure of the series is intact.

Table 3: Descriptive statistics of the 50,000-row sample (last column: all valid rows)

Measurement	Mean	Std. dev.	Min	Max	Full-data mean
Global active power (kW)	1.678	1.348	0.194	9.272	1.092
Global reactive power (kW)	0.136	0.119	0.000	0.862	0.124
Voltage (V)	241.032	3.578	226.320	251.700	240.840
Global intensity (A)	7.099	5.673	0.800	40.400	4.628
Sub-metering 1 (Wh)	1.127	6.249	0.000	77.000	1.122
Sub-metering 2 (Wh)	1.904	7.733	0.000	78.000	1.299
Sub-metering 3 (Wh)	7.205	8.617	0.000	20.000	6.458

The sample mean active power (1.678 kW) is clearly higher than the mean over the full cleaned dataset (1.092 kW), which is expected for a winter holiday period with heating, lighting and cooking. This is an important caveat for interpretation: the results describe this window and should not be read as a statement about the household in other seasons.

3.4 Exploratory analysis

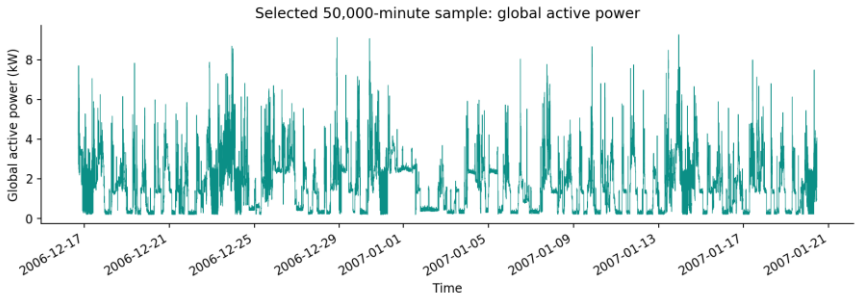


Figure 1: Global active power over the selected five-week sample

Figure 1 shows a signal made of a low base load, with daily rhythm, punctuated by short high-power events. The series is non-stationary, with clear bursts and quiet periods, which is typical of household load where individual appliances create abrupt steps.

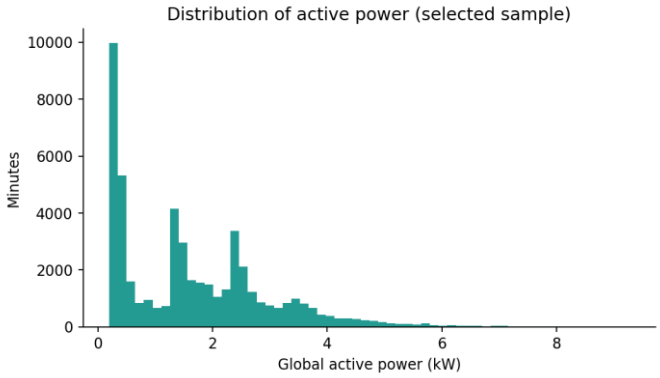


Figure 2: Distribution of global active power in the sample

The distribution in Figure 2 is right-skewed: the median is 1.432 kW against a mean of 1.678 kW and a maximum of 9.272 kW. Large values are rare but dominate squared-error metrics, which is one reason RMSE and MAE can rank models differently.

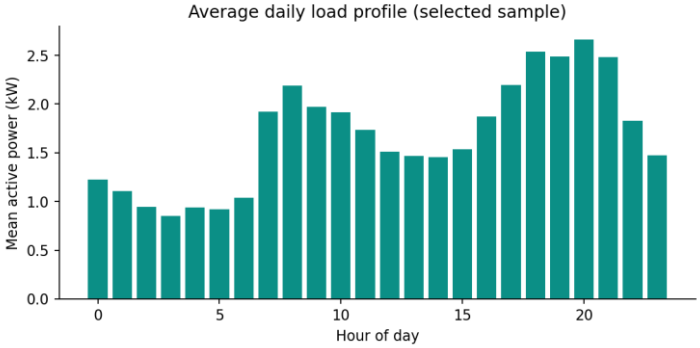


Figure 3: Average load by hour of day

Mean consumption varies systematically through the day (Figure 3). This supports including cyclic hour-of-day features, so that the network can learn the typical level expected at a given time.

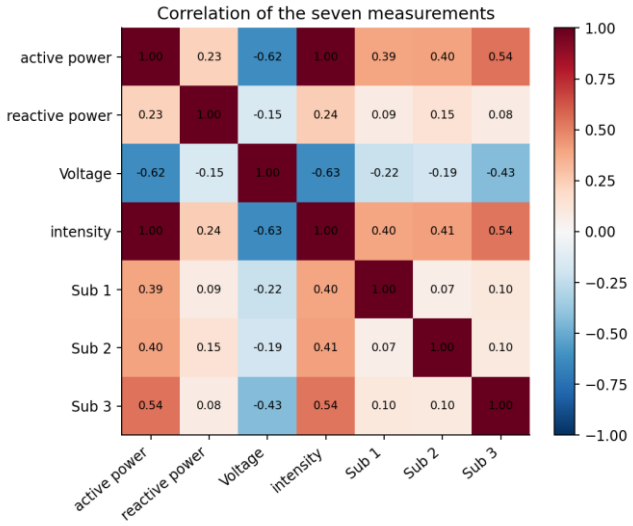


Figure 4: Correlation matrix of the seven measurements

Figure 4 shows that global active power and global intensity are almost perfectly correlated (about 1.00), as expected from the physical relationship between power and current at near-constant voltage. Voltage is negatively correlated with power (−0.62): when the household draws more current, the supply voltage sags. Sub-meter 3 (water heater and air conditioning) has the strongest link to total power among the sub-meters (0.54). The lag-1 autocorrelation of active power is 0.95 while the lag-30 autocorrelation is 0.64, which motivates using both a very recent and a longer-range lag as inputs.

4. Methodology

4.1 Supervised formulation

For every minute t that has the required history, the input vector is built only from readings recorded before t and from calendar information known at t, and the target is the active power observed at t. The model therefore answers the question “what will the next minute’s active power be?” given what has just happened. The network does not see the value it is asked to predict.

4.2 Feature engineering

Twelve features are used. Six are the non-target electrical readings one minute earlier, two are active-power lags, and four are cyclic encodings of the time of day and the day of the week.

Table 4: The twelve input features

Feature	Meaning
Global_reactive_power_lag_1	Reactive power at t − 1 min
Voltage_lag_1	Voltage at t − 1 min
Global_intensity_lag_1	Current intensity at t − 1 min
Sub_metering_1_lag_1	Kitchen sub-meter at t − 1 min
Sub_metering_2_lag_1	Laundry sub-meter at t − 1 min
Sub_metering_3_lag_1	Water-heater / air-conditioner sub-meter at t − 1 min
Global_active_power_lag_1	Active power at t − 1 min (persistence value)
Global_active_power_lag_30	Active power at t − 30 min (longer-range context)
hour_sin, hour_cos	Cyclic encoding of the time of day

Feature	Meaning
weekday_sin, weekday_cos	Cyclic encoding of the day of the week

Hour and weekday are cyclic: 23:59 is close to 00:00 and Sunday is close to Monday. Encoding each as a sine–cosine pair places these neighbours close together in feature space. With h the time of day in hours (including the minute fraction) and d the weekday index (0 to 6):

$$hour_sin = sin(2\pi h / 24),\ \ hour_cos = cos(2\pi h / 24)$$

$$weekday_sin = sin(2\pi d / 7),\ \ weekday_cos = cos(2\pi d / 7)$$

4.3 Leakage prevention and exact lag matching

Lag features are looked up by exact timestamp rather than by row offset. For a target at time t, the code reads the readings at exactly t − 1 minute and t − 30 minutes. If either timestamp is absent because a reading was missing, that sample is discarded instead of being filled in. This guarantees that a "30-minute lag" really is 30 minutes and avoids silently shifting the history after a gap. Starting from 50,000 rows this leaves 49,962 supervised samples.

4.4 Chronological train, validation and test split

The supervised samples are kept in time order and cut into three consecutive blocks: the first 70% for training, the next 15% for validation and early stopping, and the final 15% as an untouched test set. No random shuffling is used at any point.

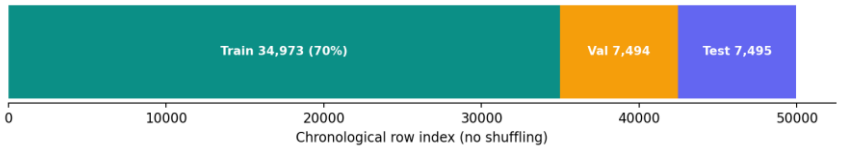


Figure 5: Chronological partitioning of the supervised samples

Table 5: Partition sizes

Partition	Share	Samples	Period
Training	70%	34,973	earliest
Validation	15%	7,494	middle
Test	15%	7,495	2007-01-15 to 2007-01-20

Figure 5 and Table 5 show the resulting partitions. The test block covers 15 to 20 January 2007, about five days, which no part of training or model selection has seen.

4.5 Feature scaling

Neural networks train more reliably when inputs are on comparable scales; here voltage is around 240 whereas reactive power is around 0.1. A standardising scaler (zero mean, unit variance) is fitted on the training samples only and then applied unchanged to the validation and test samples. Fitting on training data alone prevents information about the later periods from leaking into the preprocessing. The fitted scaler is saved with the model so that the dashboard applies identical scaling to new inputs.

4.6 Network architecture

The architecture is shown in Figure 6 and summarised layer by layer in Table 6.

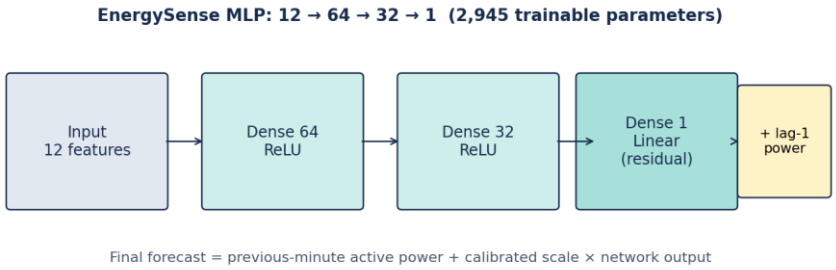


Figure 6: Architecture of the EnergySense multilayer perceptron

Table 6: Layer structure and parameter count

Layer	Output size	Activation	Parameters
Input	12	—	0
Dense 1	64	ReLU	832
Dense 2	32	ReLU	2,080
Dense 3 (output)	1	Linear	33
Total			2,945

The network is intentionally small. With about three thousand parameters and roughly 35,000 training samples it is unlikely to memorise the data, it trains in seconds on a CPU, and it is easy to inspect. The first layer has 12 × 64 + 64 = 832 parameters, the second 64 × 32 + 32 = 2,080 and the output layer 32 + 1 = 33.

4.7 Residual formulation and validation calibration

Rather than predicting the absolute power directly, the network is trained to predict the **change** from the previous minute. Writing y_{t-1} for the lag-1 active power, the training target is the residual:

$$r_t = y_t - y_{t-1}$$

The final forecast adds a scaled version of the network output to the persistence value:

$$\hat{y}_t = y_{t-1} + s \cdot \hat{r}_t$$

This design has two advantages. First, a network that outputs zero reproduces the persistence forecast exactly, so the ANN can only help by learning a useful correction and the comparison with the baseline is direct. Second, the network does not have to learn the identity mapping from the lag-1 input to the output, a task that is easy in principle but wastes capacity.

The scalar s controls how strongly the learned correction is trusted. It is fitted by least squares on the validation set only and clipped to the range [0, 1.5]:

$$s = clip\left(\Sigma\, \hat{r}_i\, v_i \,/\, \Sigma\, \hat{r}_i^{\,2}\,,\ 0,\ 1.5 \right), \quad v_i = y_i - y_{i-1}$$

The fitted value is s = 0.9008, slightly below one, meaning the validation data asked for the network's correction to be shrunk by about ten percent. Using validation data for this calibration keeps the test set completely untouched.

4.8 Training configuration

Table 7: Training hyperparameters

Setting	Value
Framework	TensorFlow / Keras (Sequential model)
Optimiser	Adam, default Keras learning-rate settings
Loss / tracked metric	Mean squared error / mean absolute error
Batch size	128
Maximum epochs	100
Early stopping	Monitor validation loss, patience 12, restore best weights
Shuffling during training	Disabled (time order preserved)
Random seed	42 (with deterministic operations requested)
Epochs actually run / best epoch	33 / 21

4.9 Evaluation protocol

After training, the model produces forecasts for every test minute. The same test minutes are scored for the persistence baseline, which uses the lag-1 active power directly. MAE, RMSE and R^2 are computed for both. In addition to these headline figures, the analysis in Section 5 examines the residual distribution, the quantiles of the absolute error, the share of minutes on which each method is closer to the truth, and the error by hour of day.

5. Results and Analysis

5.1 Training behaviour



Figure 7: Training and validation loss (MSE) per epoch

Figure 7 shows the learning curves. The loss is the mean squared error of the residual target. Training loss falls steadily from 0.203 in the first epoch to 0.145 in the last. Validation loss improves more quickly at first and reaches its minimum of 0.195 at epoch 21; it then drifts slightly upward while training loss continues to fall. Early stopping ended training at epoch 33, twelve epochs after the best epoch, and restored the weights from epoch 21.

Table 8: Selected points of the training history

Epoch	Train loss	Train MAE	Val. loss	Val. MAE
1 (first)	0.2030	0.2374	0.2273	0.2590

Epoch	Train loss	Train MAE	Val. loss	Val. MAE
21 (best, restored)	0.1531	0.1865	0.1955	0.2020
33 (last)	0.1454	0.1813	0.2015	0.2033

The gap between training and validation loss is expected here for two reasons: the network has fitted the training period, and the validation block comes from a later stretch of time with its own usage pattern. The widening of the gap after epoch 21 is the signal that early stopping was designed to catch.

5.2 Headline test results

Table 9: Test-set performance on 7,495 held-out minutes

Model	MAE (kW)	RMSE (kW)	R^2
ANN (EnergySense)	0.159626	0.371657	0.898419
Persistence baseline	0.140831	0.384987	0.891002
Difference (ANN − baseline)	0.018795	-0.013329	0.007417

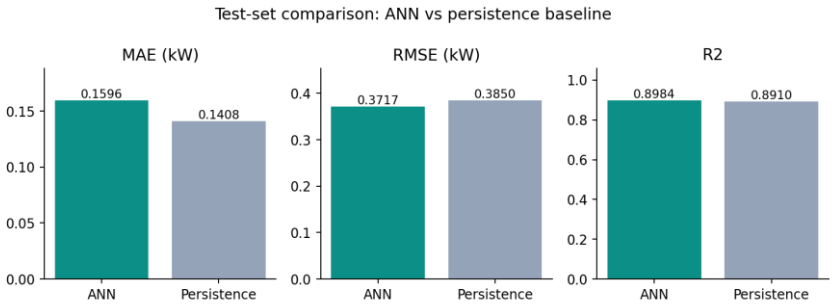


Figure 8: MAE, RMSE and R^2 for the ANN and the persistence baseline

The result is **mixed**, and the report states it that way. The ANN has a lower RMSE (0.3717 kW against 0.3850 kW, a reduction of 3.46%) and a higher R^2 (0.8984 against 0.8910). The persistence baseline, however, has the lower MAE (0.1408 kW against 0.1596 kW), so the ANN's MAE is 13.35% higher. The ANN is therefore not better on every metric, and the evidence does not support describing it as simply "more accurate" than the baseline. The sections that follow explain where this pattern comes from.

5.3 Actual versus predicted values

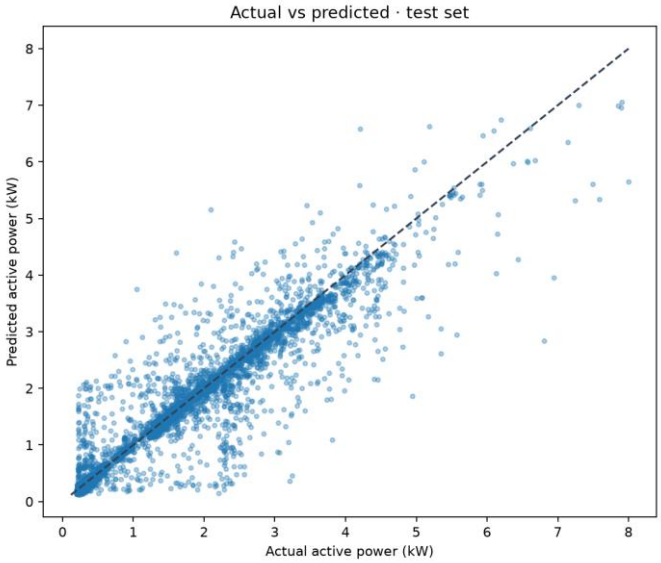


Figure 9: Actual versus ANN-predicted active power on the test set

If forecasts were perfect, all points in Figure 9 would lie on the dashed diagonal. The cloud follows the diagonal closely, with a Pearson correlation between actual and predicted values of about 0.95, and is tightest at low and medium load where most minutes lie. The scatter widens above roughly 3 kW and shows both under- and over-prediction of the largest peaks. Vertical streaks at low actual values correspond to minutes when an appliance had just switched off but the forecast, anchored on the previous high reading, remained high; horizontal streaks are the reverse, where the load jumped up unexpectedly. These events cannot be anticipated from the previous minute alone and are the main source of error for any one-minute-ahead model.

5.4 Residual analysis

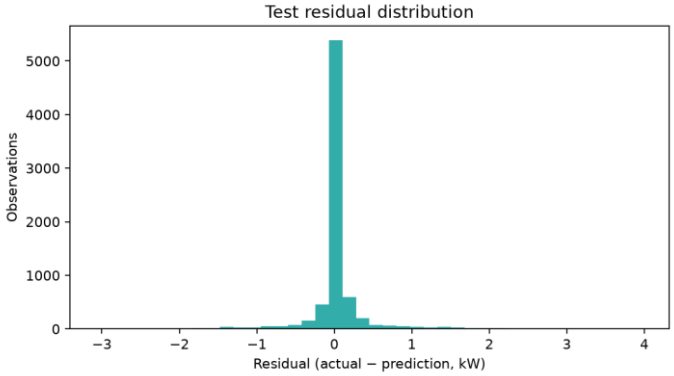


Figure 10: Distribution of test residuals (actual − prediction)

Table 10: Residual summary (actual − forecast, kW)

Statistic	ANN	Persistence	Comment
Mean	0.0303	0.0002	ANN slightly under-predicts
Standard deviation	0.3704	0.3850	ANN lower
Minimum	−3.0604	−3.1620	Largest over-prediction

Statistic	ANN	Persistence	Comment
Maximum	3.9689	3.9860	Largest under-prediction

Figure 10 shows residuals concentrated sharply around zero with long tails on both sides, the shape expected from a signal whose changes are mostly small but occasionally large. The ANN residuals have a small positive mean (0.030 kW), meaning that on average the ANN under-predicts slightly, whereas the persistence residuals are almost exactly centred (0.0002 kW). The ANN has the smaller standard deviation (0.370 against 0.385 kW), consistent with its lower RMSE.

5.5 Forecasts over time

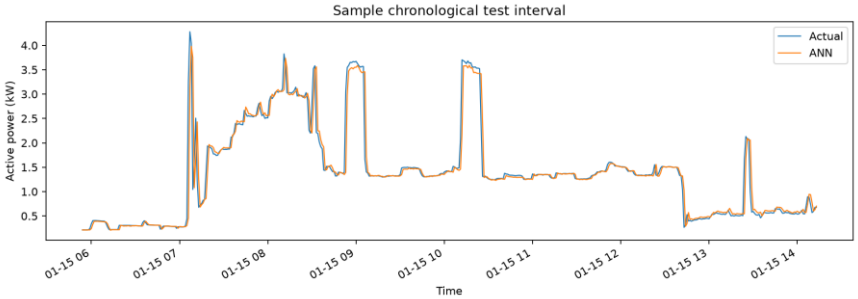


Figure 11: Actual and ANN forecast over a chronological sample of the test period

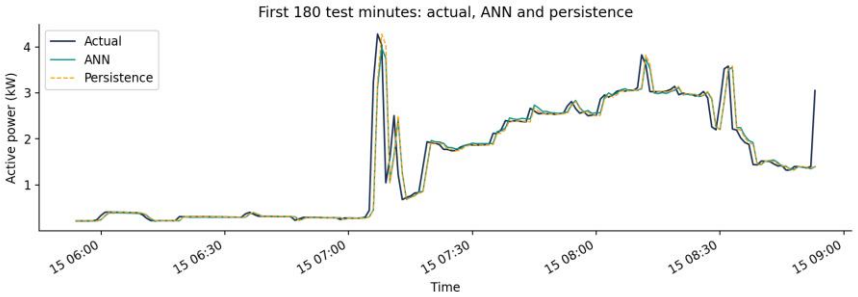


Figure 12: Detail of the first 180 test minutes with the persistence forecast added

Figure 11 shows the first 500 test minutes. The forecast follows the actual curve closely, including the timing of rises and falls, which is expected from a model that is anchored on the last observed value. Figure 12 zooms into the first three hours and adds the persistence forecast. The two forecasts are nearly indistinguishable during stable periods and both lag the actual signal by one minute at sudden steps; neither can foresee the step before it happens. The ANN differs most from persistence around transitions, where its learned correction occasionally moves the forecast towards the new level.

5.6 Distribution of absolute error

Table 11: Quantiles of the absolute error on the test set (kW)

Quantile	ANN	Persistence
Median (50%)	0.0441	0.0140
75th percentile	0.1080	0.0680
90th percentile	0.4067	0.3900
95th percentile	0.8770	0.9280
99th percentile	1.7350	1.8560

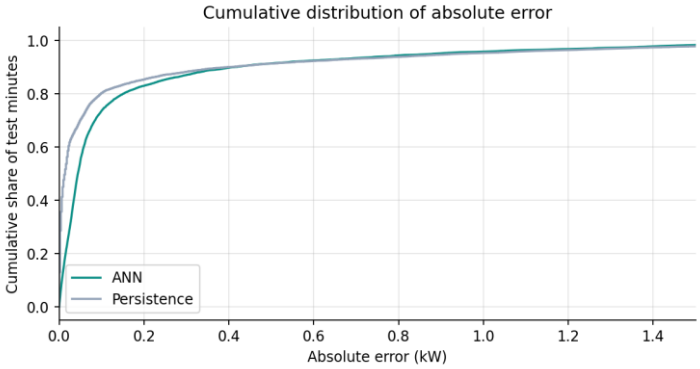


Figure 13: Cumulative distribution of absolute error for the two forecasters

Table 11 and Figure 13 explain the mixed headline result. On a typical minute, persistence is closer: its median absolute error is 0.014 kW against 0.044 kW for the ANN, and the ANN is the more accurate forecaster on only 27.4% of the test minutes. Most of the time the load barely changes, the previous reading is almost exactly right, and the ANN's small corrections add a little noise. This drives the higher ANN MAE. In the upper tail the ordering reverses: at the 95th and 99th percentiles the ANN's absolute errors are smaller than persistence (0.877 against 0.928 kW and 1.735 against 1.856 kW). The ANN therefore reduces the size of the largest errors, which are weighted most heavily by squared-error metrics, and this is why it wins on RMSE and R^2.

5.7 Error by hour of day

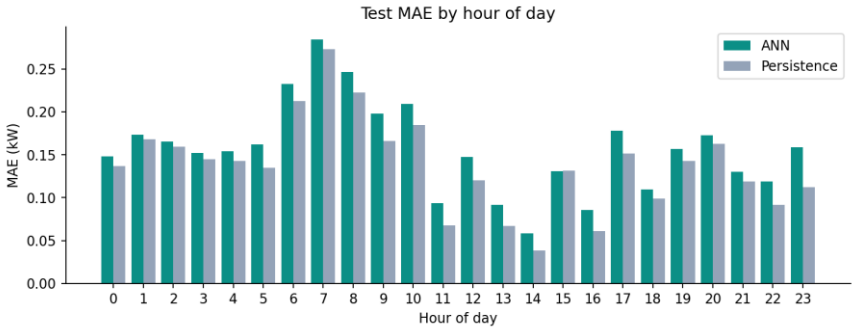


Figure 14: Mean absolute error of both forecasters by hour of day

Error is not uniform through the day (Figure 14). Both forecasters are least accurate in the morning hours (06:00 to 10:00), when appliances such as heating and cooking equipment switch on and off frequently, and most accurate in the early afternoon, when the household load is low and stable. The persistence baseline has the lower MAE in most hours, which is consistent with the overall MAE result. Only a few hours are close to a tie, such as 15:00. The hour-of-day breakdown is based on 300 to 360 minutes per hour, so individual hourly differences should be read as indicative rather than precise.

5.8 Summary of findings

- The ANN improves RMSE by 3.46% and R^2 by 0.0074 over persistence.
- The ANN has a 13.35% higher MAE than persistence and is closer to the truth on only 27.4% of test minutes.
- The ANN reduces the largest errors (95th and 99th percentiles), but adds small errors in calm periods.

- Both models perform best in the early afternoon and worst in the morning, and both fail to anticipate abrupt load steps.
- Because the network is anchored on the lag-1 value and the series is highly autocorrelated (0.95), the achievable gain over persistence at a one-minute horizon is modest.

6. System Implementation

6.1 Software stack and project structure

The project is written in Python. Data handling uses pandas and NumPy, scaling and metrics use scikit-learn, the network is built and trained with TensorFlow/Keras (Abadi et al., 2016), plots are made with Matplotlib, and the interface is built with Streamlit. The repository is organised so that each stage of the workflow is a separate script and the shared logic lives in one module.

Table 12: Repository contents

File / folder	Purpose
preprocess.py	Inspects the raw file and writes the chronological 50,000-row sample and dataset summary
train_model.py	Builds supervised data, fits the scaler, trains the MLP, calibrates the residual scale, saves artifacts
evaluate.py	Computes MAE, RMSE and R^2, draws the four evaluation plots and writes the generated report
src/energy_sense.py	Loading, cleaning, exact-lag feature engineering and prediction utilities shared by all scripts and the app
app.py	Streamlit dashboard (Dashboard, Prediction, Data Explorer, About)
models/	Saved Keras model, feature scaler and training metadata
outputs/	Test predictions, metrics, training history and plots
tests/	Unit tests for loading, cleaning, lag construction and prediction validation

6.2 The shared feature module

All feature logic is concentrated in src/energy_sense.py so that training, evaluation and the live dashboard cannot drift apart. The same list of feature names defines the model input, the same lookup-by-timestamp function builds training samples, and a dedicated function validates and converts user-entered readings into a single feature row for prediction. Sharing this code is a deliberate protection against train/serve skew, a common source of silent errors in deployed models.

6.3 The Streamlit dashboard

The dashboard is deployed on Streamlit Community Cloud and presents the work through four pages, selected from a sidebar. It uses a light, consistent visual theme with metric cards and charts.

- **Dashboard.** Shows dataset counts (source readings, complete readings, model sample, number of input features), the experiment window, the ANN-versus-persistence MAE and RMSE chart, a short project snapshot and the four evaluation images.
- **Prediction.** Lets a user enter, edit or upload as CSV the most recent 30 consecutive one-minute readings and press a button to obtain the next-minute estimate of active power. Inputs are

validated (all six measurements, finite numbers, oldest first) and clear messages are shown for missing columns, short files or invalid entries.

- **Data Explorer.** Provides a date-range slider, a selectable measurement chart, quick statistics, a scrollable table of the observations and a download button for the filtered rows.
- **About.** Summarises the model workflow and states the responsible-use notes.

The application degrades gracefully: if the model files or dataset artifacts are missing it shows an explanatory message instead of failing, and the model is loaded once and cached so that repeated predictions are fast.

6.4 Prediction pipeline

When the user presses the estimate button, the dashboard converts the 30 readings to a feature row using the same function used in training. The lag-1 features come from the newest row, the 30-minute active-power lag comes from the oldest row, and the hour and weekday encodings come from the optional forecast timestamp (the current time is used if none is entered). The row is scaled with the saved scaler, the network produces a residual, and the final estimate is the newest active-power reading plus the validation-calibrated scale times that residual. Because the model uses only the oldest and newest rows of the 30-row window for its active-power lags, the intermediate rows are validated for completeness but do not otherwise change the estimate.

6.5 Testing and reproducibility

A suite of unit tests covers parsing of the raw format and the missing-value marker, cleaning and exact-timestamp lag construction on a synthetic series, and validation and assembly of the prediction input. All random seeds are fixed to 42 and deterministic operations are requested, so training can be repeated on the supplied sample. The commands to reproduce the pipeline are listed in Appendix A.

6.6 Dashboard screenshots

The following screenshots were captured from the dashboard (app.py) running with the trained model and evaluation artifacts from the project repository. They show the four pages described above.

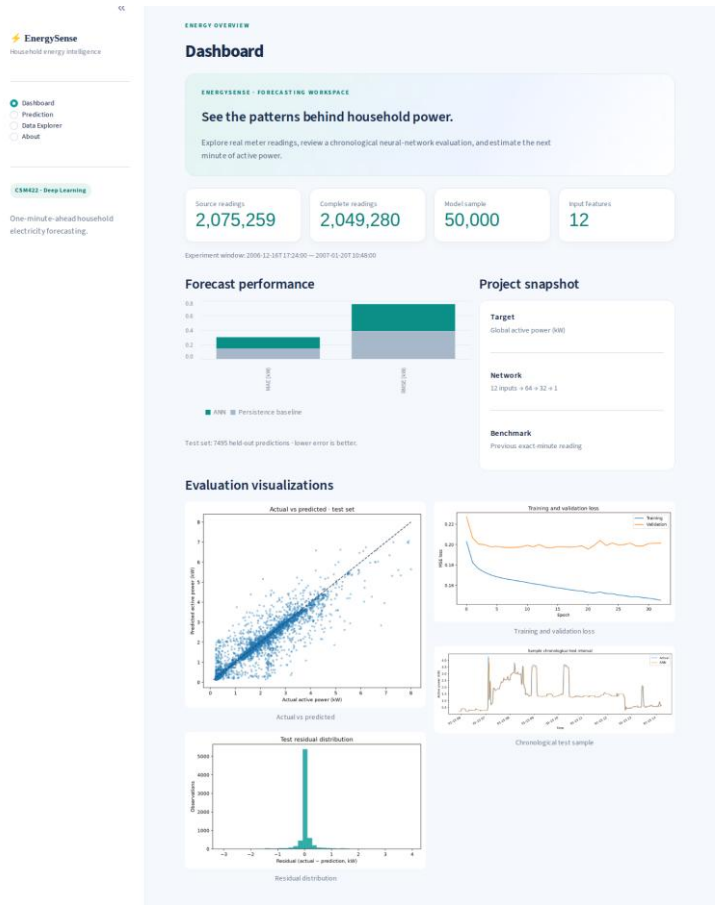


Figure 15: Dashboard page: dataset counts, forecast-performance chart and the four evaluation plots

In Figure 15 the forecast-performance chart draws the ANN and persistence values as stacked segments of one bar, so the two values should be read as the height of each coloured segment and not as the total bar height.

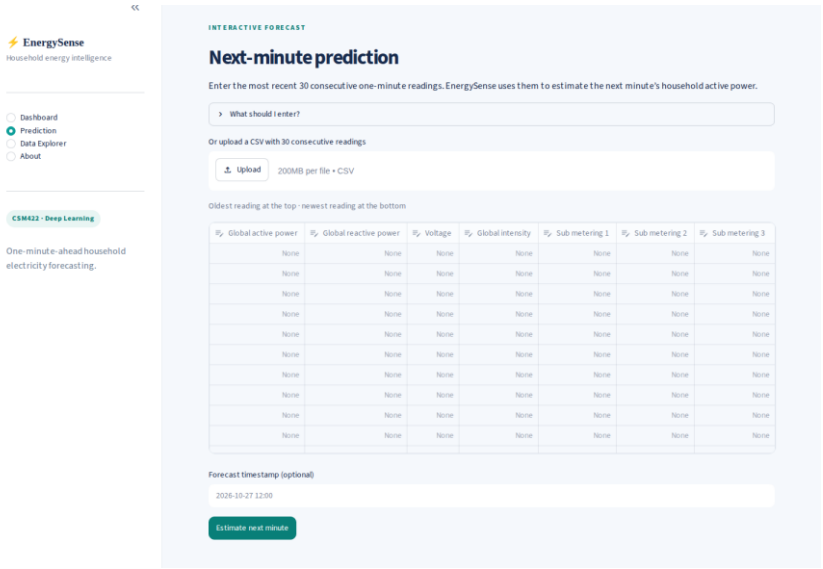


Figure 16: Prediction page: 30-row input table, CSV upload and the estimate button

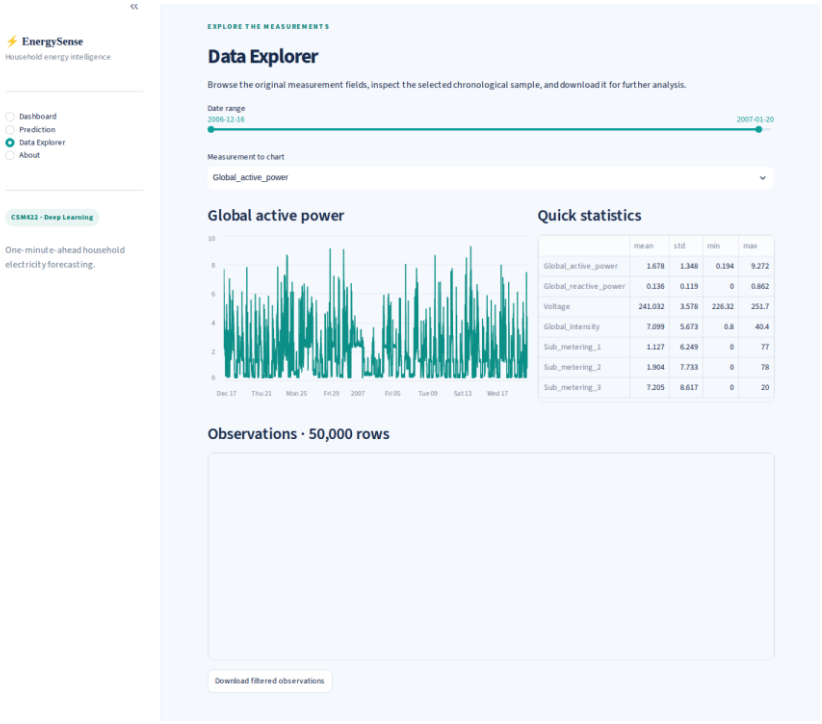


Figure 17: Data Explorer page: date range, measurement chart, statistics and table

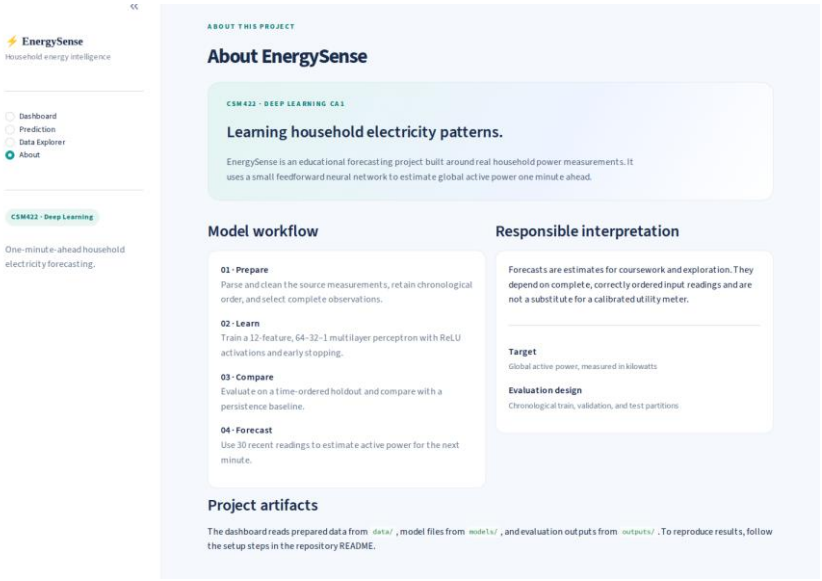


Figure 18: About page: model workflow and responsible-interpretation notes

7. Discussion

7.1 Interpreting the mixed result

The most informative observation is that persistence is both the baseline and the anchor of the ANN. At a one-minute horizon the series changes little most of the time, so the previous reading is a nearly perfect forecast for the majority of minutes. A learned correction can only add value when a change is predictable from the other inputs, and such predictable changes are uncommon. When the correction is applied everywhere, it slightly perturbs the many accurate forecasts (raising MAE) while helping on the rarer large moves (lowering RMSE). The validation-calibrated scale of 0.90 shows that the data itself asked for the correction to be damped.

This result should not be read as a failure. It is the expected behaviour of a compact model on a highly autocorrelated target, and reporting it transparently is more valuable than selecting only the metric on which the ANN wins. The practical conclusion is that the ANN offers a small, real improvement in the metric that penalises large errors, at the cost of a small deterioration in typical-minute accuracy. Which trade-off is preferable depends on the application: for peak-load awareness the RMSE gain matters more, whereas for average-error reporting persistence is preferable.

7.2 Limitations

- **Single household and short window.** All results come from one home and a five-week winter period. The test block spans about five days, so conclusions may not transfer to other seasons, households or years.
- **Small test set and single run.** The test set has 7,495 minutes that are strongly autocorrelated, so the effective sample size is smaller than the row count suggests. Only one seed and one split were evaluated, and no confidence intervals are reported.
- **Limited baselines.** The ANN is compared only with persistence. Linear regression, gradient-boosted trees or recurrent networks were not evaluated and might perform differently.
- **Compact feature set and no tuning.** The architecture, learning rate and lag choices (1 and 30 minutes) were fixed in advance. No systematic hyperparameter search was run.
- **Dashboard input window.** The live predictor asks for 30 readings but uses only the oldest and newest of them, so it exploits less of the supplied history than the prompt may suggest.

7.3 Threats to validity

Leakage was controlled by exact-timestamp lags, training-only scaling and chronological splitting, and the residual scale was fitted on validation data only. Remaining threats are mainly statistical: results from one split can differ from those of another time block, and the drift between the training and validation periods visible in Section 5.1 suggests that load patterns change over time.

7.4 Ethical and privacy considerations

Fine-grained household load data can reveal when residents are at home, asleep or using particular appliances. Even though the dataset used here is public and anonymous, any deployment on real customers' data would need informed consent, data minimisation and secure storage. The dashboard states that its forecasts are estimates for coursework and exploration and are not a substitute for a calibrated utility meter, and the derived data included in the public repository should be shared only with the necessary permissions.

8. Conclusion and Future Work

8.1 Conclusion

EnergySense demonstrates a complete deep-learning workflow on a real household electricity dataset: careful cleaning without imputation, leakage-free exact-lag features, a chronological split, a compact two-hidden-layer network trained with Adam and early stopping, a residual formulation anchored on persistence, evaluation against a baseline, and a deployed dashboard.

On 7,495 held-out minutes the ANN reached an RMSE of 0.3717 kW and R^2 of 0.8984, improving on persistence (0.3850 kW and 0.8910), but its MAE of 0.1596 kW was higher than the baseline's 0.1408 kW. Diagnostics show that the ANN reduces the largest errors while being slightly less accurate than

persistence on typical minutes. The honest summary is a modest, metric-dependent gain rather than a uniform improvement.

8.2 Future work

- Evaluate several temporal cut points (rolling-origin evaluation) and multiple seeds to quantify uncertainty with confidence intervals.
- Add stronger baselines, such as linear regression, gradient-boosted trees, ARIMA and recurrent or convolutional sequence models.
- Use the full 2 million-row dataset, covering all four years and all seasons, and add calendar features such as holidays.
- Tune the architecture, regularisation and lag set systematically, and test longer horizons (5, 15 and 60 minutes) where learned models usually gain more over persistence.
- Train with a loss that better balances typical and extreme minutes, for example a Huber or quantile loss, and produce prediction intervals.
- Make the dashboard use the full 30-minute window, for instance with a sequence model, and add anomaly highlighting.

References

Abadi, M. et al. (2016). TensorFlow: A system for large-scale machine learning. In Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI).

Goodfellow, I., Bengio, Y. and Courville, A. (2016). Deep Learning. MIT Press.

Hebrail, G. and Berard, A. (2006). Individual Household Electric Power Consumption. UCI Machine Learning Repository.

Hyndman, R. J. and Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.). OTexts.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In International Conference on Learning Representations (ICLR).

McKinney, W. (2010). Data structures for statistical computing in Python. In Proceedings of the 9th Python in Science Conference.

Nair, V. and Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. In Proceedings of the 27th International Conference on Machine Learning (ICML).

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Prechelt, L. (1998). Early stopping — but when? In Neural Networks: Tricks of the Trade. Springer.

Rumelhart, D. E., Hinton, G. E. and Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536.

Appendix A: Reproduction Commands

Python 3.10 or 3.11 is recommended. The original household_power_consumption.txt must be in the project root for preprocessing; training and evaluation can run from the included selected sample.

```
py -3.11 -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
python preprocess.py
python train_model.py
python evaluate.py
python -m unittest discover -s tests -v
streamlit run app.py
```

Project links:

- Code repository: https://github.com/Kartik070704/Deep-Learning
- Deployed dashboard: https://deep-learning-2q62ytadkeqmj3cjkotrun.streamlit.app/

Appendix B: Key Code Excerpts

B.1 Exact-timestamp lag features (src/energy_sense.py)

```
def make_supervised(clean):
    frame = clean.sort_values("DateTime", kind="stable").reset_index(drop=True)
    by_time = frame.set_index("DateTime")
    times = pd.DatetimeIndex(frame["DateTime"])
    prior_1 = times - pd.Timedelta(minutes=1)
    prior_30 = times - pd.Timedelta(minutes=30)
    x = pd.DataFrame(index=frame.index)
    for col in MEASUREMENT_COLUMNS:
        if col != TARGET:
            x[f"{col}_lag_1"] = by_time[col].reindex(prior_1).to_numpy()
    x["Global_active_power_lag_1"] = by_time[TARGET].reindex(prior_1).to_numpy()
    x["Global_active_power_lag_30"] = by_time[TARGET].reindex(prior_30).to_numpy()
    hour = times.hour + times.minute / 60.0
    weekday = times.dayofweek
    x["hour_sin"] = np.sin(2 * np.pi * hour / 24)
    x["hour_cos"] = np.cos(2 * np.pi * hour / 24)
    x["weekday_sin"] = np.sin(2 * np.pi * weekday / 7)
    x["weekday_cos"] = np.cos(2 * np.pi * weekday / 7)
    valid = x.notna().all(axis=1)
```

B.2 Model definition and training (train_model.py)

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(len(FEATURE_COLUMNS),)),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(1, activation="linear"),
], name="EnergySense_MLP")
model.compile(optimizer=tf.keras.optimizers.Adam(), loss="mse", metrics=["mae"])
callback = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=12, restore_best_weights=True)
history = model.fit(scale(x_train), y_train_residual,
    validation_data=(scale(x_val), y_val_residual),
    epochs=epochs, batch_size=128, shuffle=False, callbacks=[callback])
```

B.3 Validation calibration of the residual scale

```
denominator = float(np.dot(val_residual_prediction, val_residual_prediction))
residual_scale = float(np.dot(val_residual_prediction, val_change)) / denominator
residual_scale = float(np.clip(residual_scale, 0.0, 1.5))
prediction = test_baseline + residual_scale * residual_prediction
```

Appendix C: Raw Evaluation Output

Contents of outputs/evaluation_metrics.json as produced by evaluate.py:


```
{"test_rows":7495,
"unit":"kW",
"ANN":{"MAE":0.1596260166374475,
"RMSE":0.3716572331323523,
"R2":0.8984191568615212},
"Persistence_baseline":{"MAE":0.1408306871247498,
"RMSE":0.3849865835493604,
"R2":0.8910021768750815},
"ANN_lower_MAE":false,
"ANN_lower_RMSE":true}
```